

Introduction to Agents and Multiagents enhanced by LLMs

WHO I AM

MATHEUS ANDRADE

Member of the **Conect2ai**
Research Group | Projects:
CNPq and **Mover**



Master's Candidate

Graduate Program in Electrical and
Computer Engineering
PPgEEC | UFRN



Computer Engineer
UFRN



**Bachelor of Science
and Technology**
UFRN



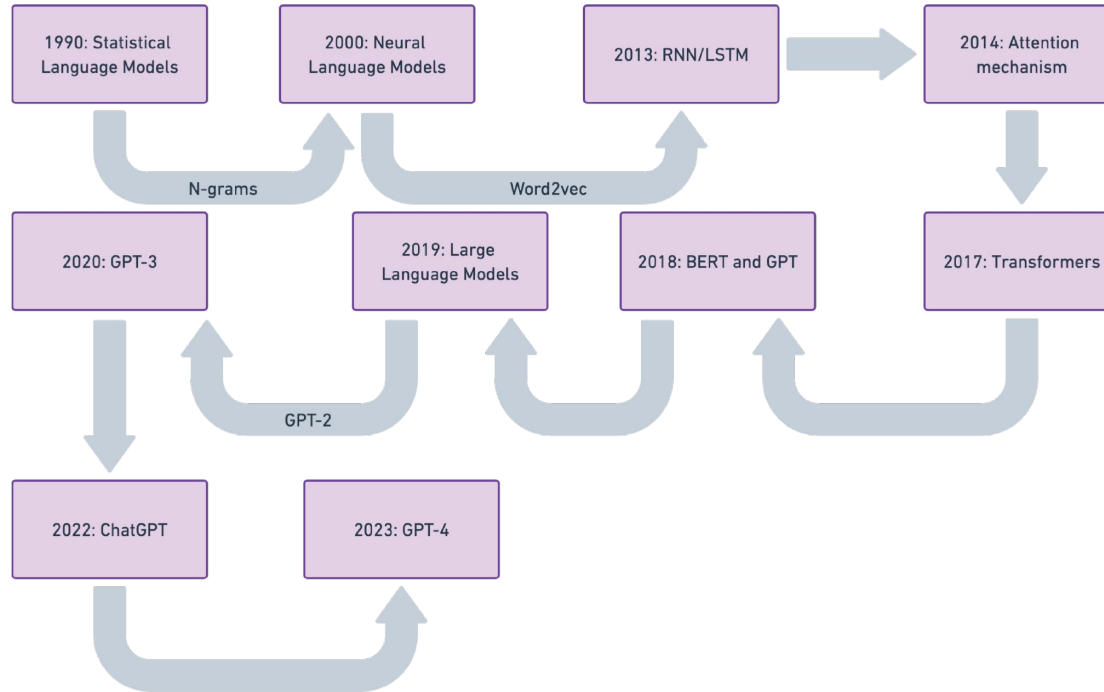
INTRODUCTION

WHAT ARE LLMS?



INTRODUCTION

A BRIEF HISTORY



Source: <https://arxiv.org/html/2408.13296v1>

WHAT ARE LLMS?

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

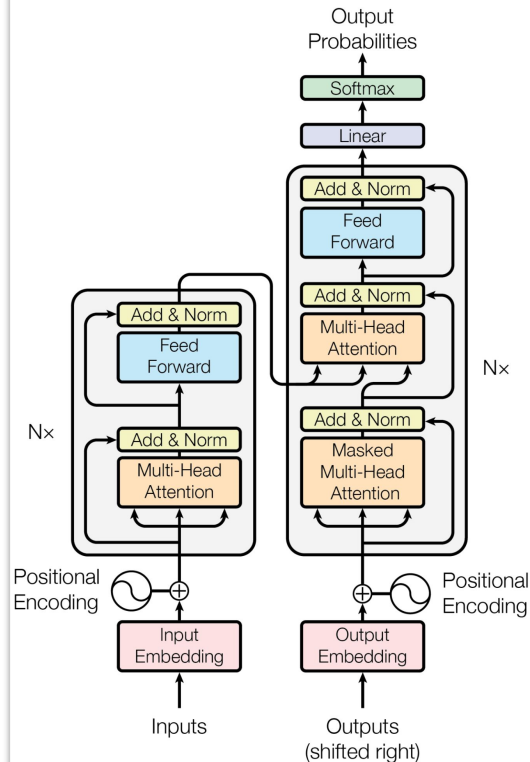
Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

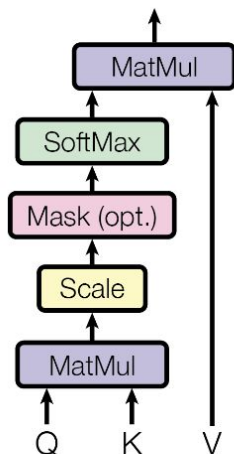
The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and



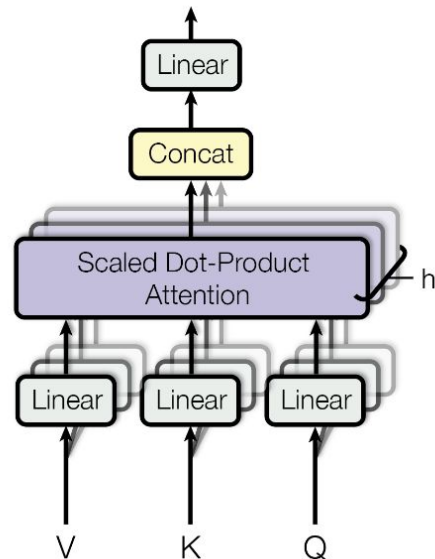
WHAT ARE LLMS?

Scaled Dot-Product Attention



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Multi-Head Attention

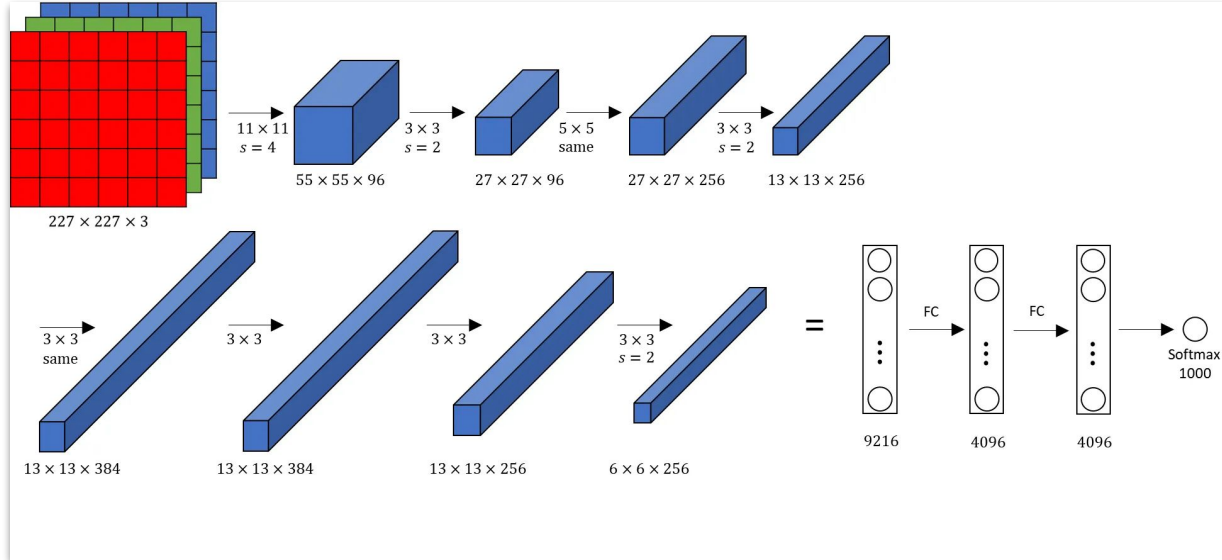


$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

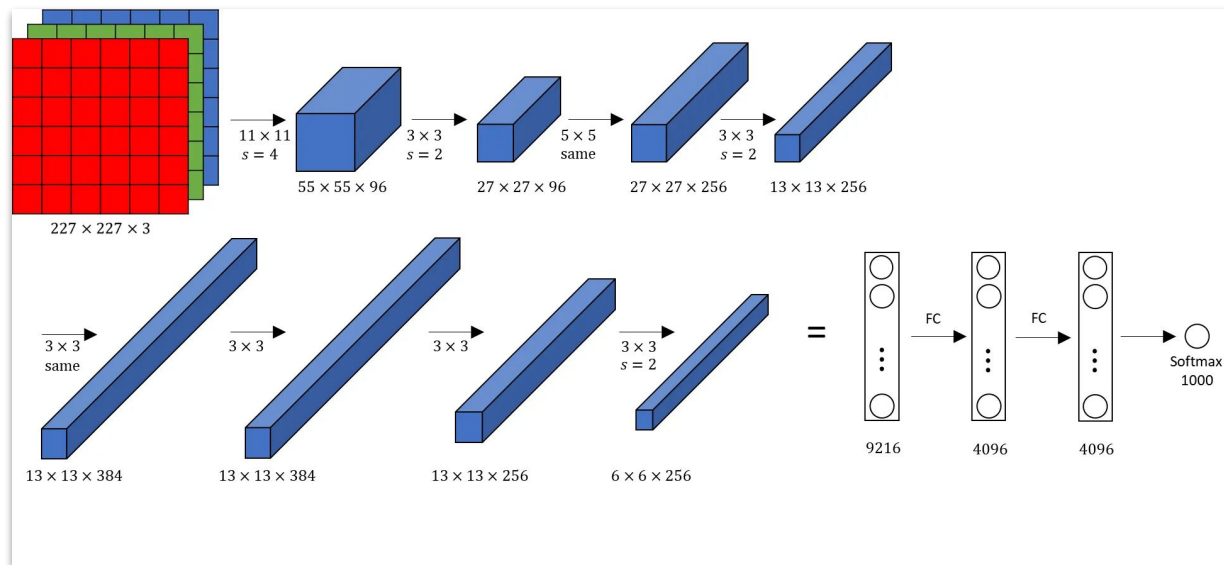
INTRODUCTION

WHAT ARE LLMS?



INTRODUCTION

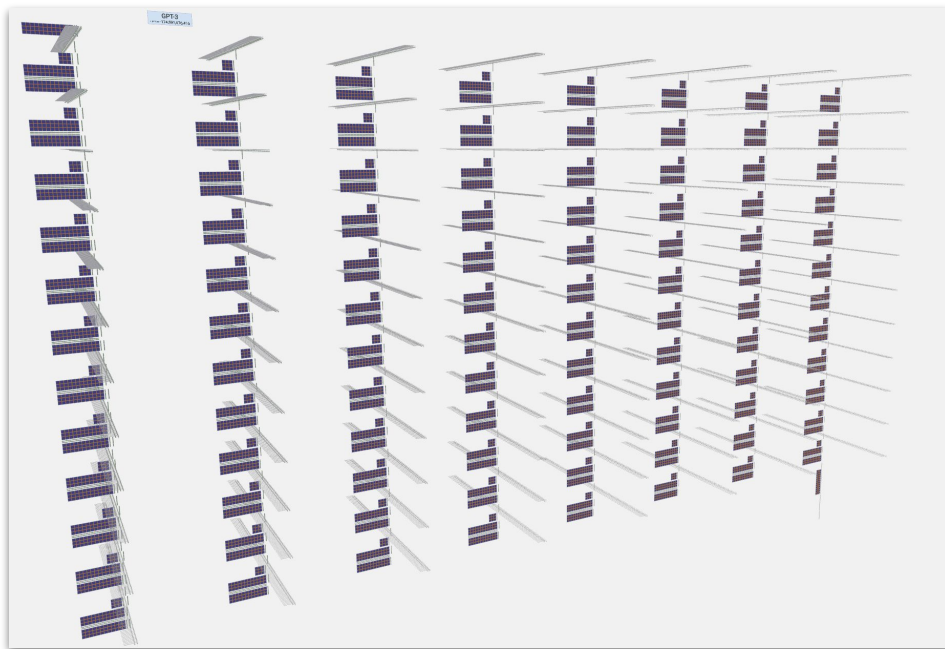
WHAT ARE LLMS?



60 M

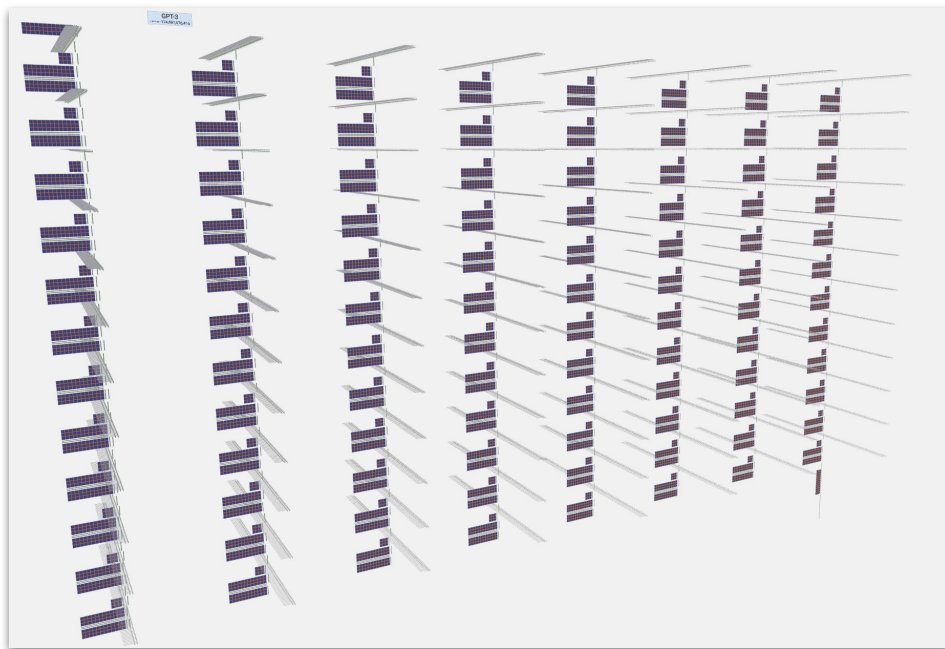
INTRODUCTION

WHAT ARE LLMS?

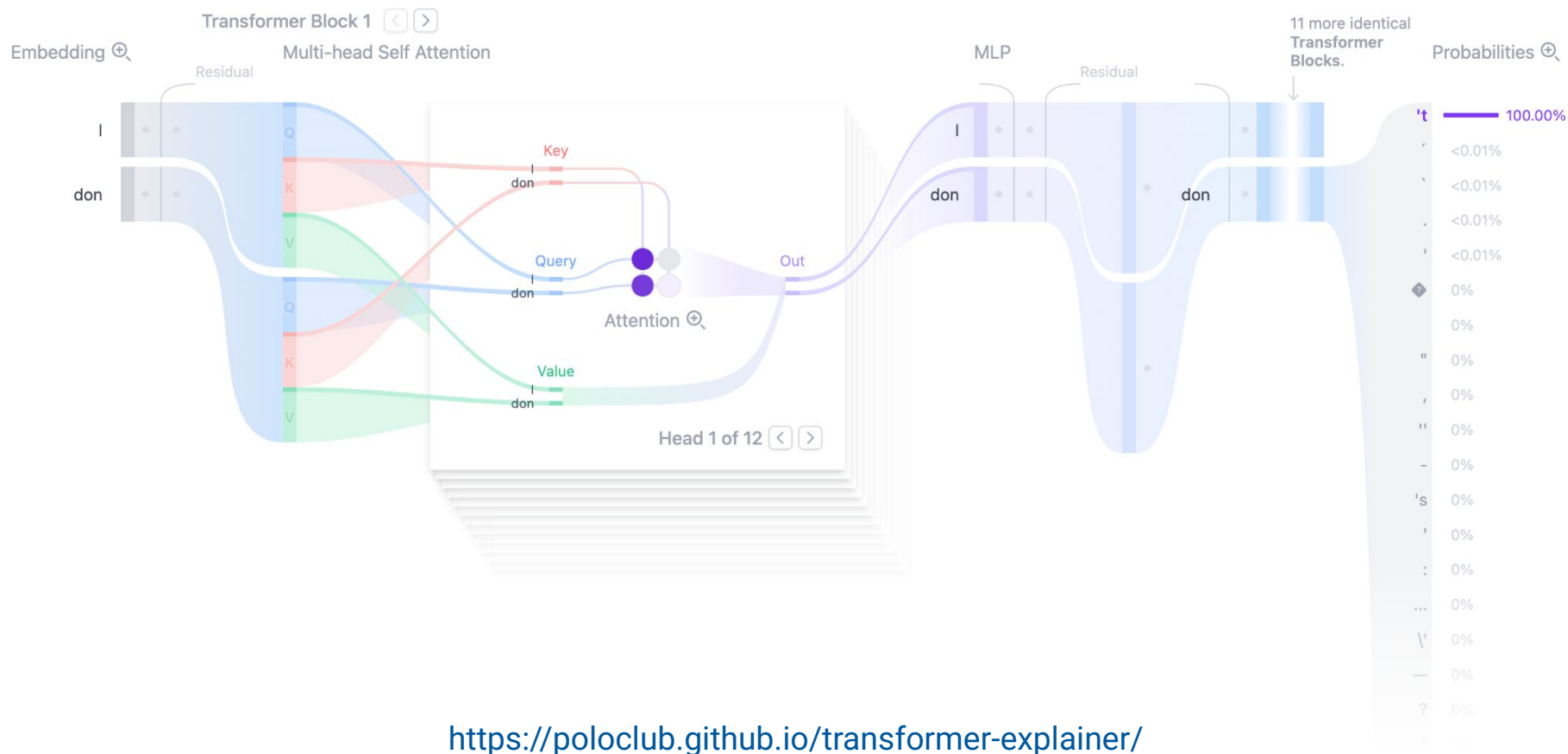


INTRODUCTION

WHAT ARE LLMS?



175 B



<https://poloclub.github.io/transformer-explainer/>

Visualize LLM Tokens

GPT2

GPT3.5 / GPT4

LLaMA

I don't know how to make LLM Agents

Tokens: 10 Characters: 35

Clear

Show example

I don't know how to make LLM Agents

<https://tokenvisualizer.netlify.app/>

SPECIAL TOKENS

```
✖ ▶ Uncaught Error: The text contains a special token that is  
not allowed: <|endoftext|>  
    at App.tsx:35:5  
    at App.tsx:75:18  
    Show ignore-listed frames
```



Hello how are you?

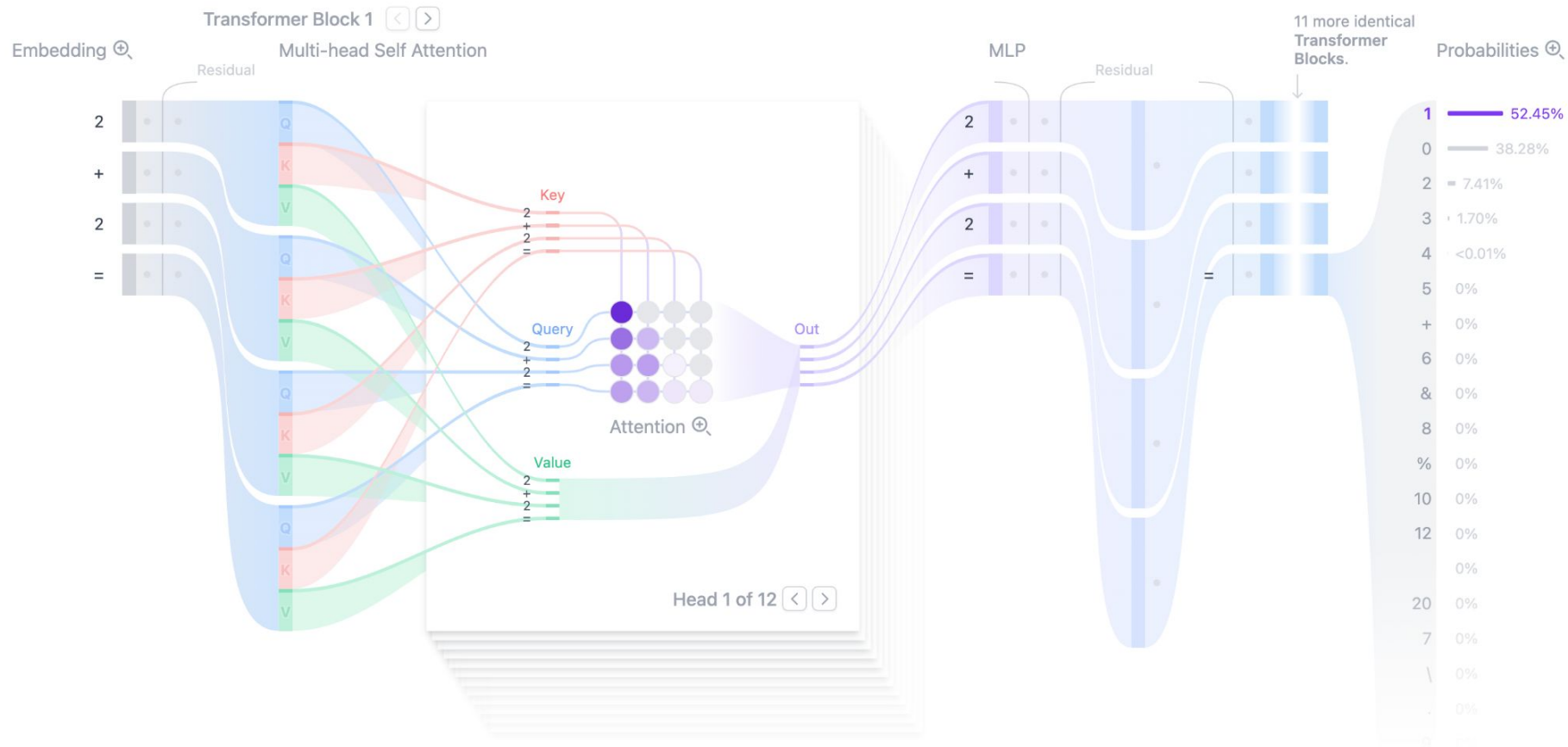
`<|begin_of_text|>`
`<|start_header_id|>user<|end_header_id|>`

Hello how are you?<|eot_id|>

Hello! I'm just a language model, so I don't have feelings or emotions like humans do, but I'm functioning properly and ready to assist you with any questions or tasks you may have. How about you? How's your day going so far?

`<|start_header_id|>assistant<|end_header_id|>`

Hello! I'm just a language model, so I don't have feelings or emotions like humans do, but I'm functioning properly and ready to assist you with any questions or tasks you may have. How about you? How's your day going so far?<|eot_id|>





LLM AGENT



WHAT IS AN AGENT?

What is $2 + 2$?

sum_tool

subtract_tool

division_tool

multiplication_tool



WHAT IS AN AGENT?

What is $2 + 2$?

sum_tool

subtract_tool

division_tool

multiplication_tool



WHAT IS AN AGENT?

What is $2 + 2$?

`sum_tool(2, 2)`



WHAT IS AN AGENT?

What is $2 + 2$?



INTRODUCTION

PILLARS

INTRODUCTION

PILLARS

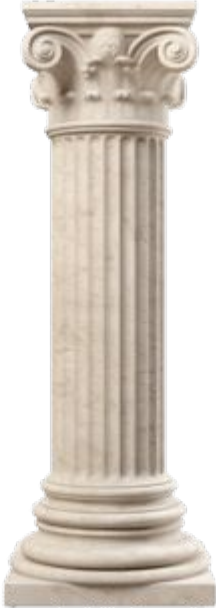
THOUGHT



INTRODUCTION

PILLARS

THOUGHT



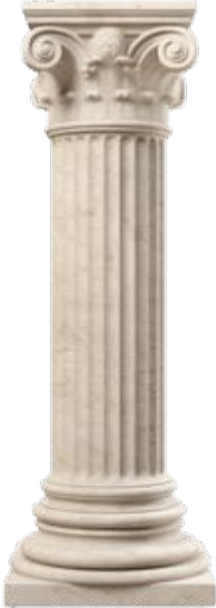
ACTION



INTRODUCTION

PILLARS

THOUGHT



ACTION



OBSERVATION

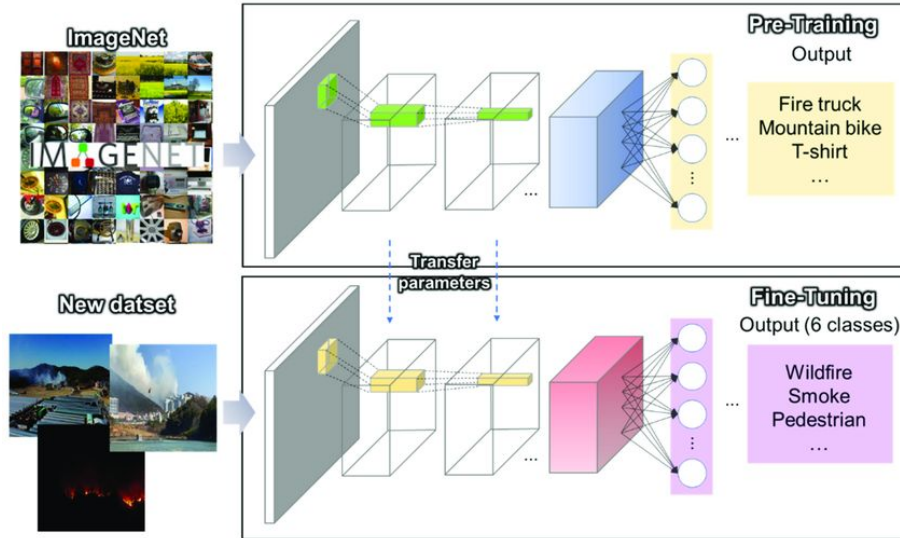


INTRODUCTION

HOW AGENTS ARE TRAINED?



FINE-TUNING



HOW AGENTS ARE TRAINED?

Function Calling

HOW AGENTS ARE TRAINED?

```
{
  "content": "You are a function calling AI model. You are provided with function signatures within <tools></tools> XML tags. You may call one or more functions to assist with the user query. Don't make assumptions about what values to plug into functions. Here are the available tools:<tools> [{ 'type': 'function', 'function': { 'name': 'get_stock_price', 'description': 'Get the current stock price of a company', 'parameters': { 'type': 'object', 'properties': { 'company': { 'type': 'string', 'description': 'The name of the company' }, 'required': [ 'company' ] } }, { 'type': 'function', 'function': { 'name': 'get_movie_details', 'description': 'Get details about a movie', 'parameters': { 'type': 'object', 'properties': { 'title': { 'type': 'string', 'description': 'The title of the movie' }, 'required': [ 'title' ] } } } ] </tools> Use the following pydantic model json schema for each tool call you will make: { 'title': 'FunctionCall', 'type': 'object', 'properties': { 'arguments': { 'title': 'Arguments', 'type': 'object', 'name': { 'title': 'Name', 'type': 'string' }, 'required': [ 'arguments', 'name' ] } } } For each function call return a json object with function name and arguments within <tool_call></tool_call> XML tags as follows:\n<tool_call>\n{tool_call}\n</tool_call>",
  "role": "system"
},
```

HOW AGENTS ARE TRAINED?

```
{
  "content": "You are a function calling AI model. You are provided with function signatures within <tools></tools> XML tags. You may call one or more functions to assist with the user query. Don't make assumptions about what values to plug into functions. Here are the available tools:<tools> [{ 'type': 'function', 'function': { 'name': 'get_stock_price', 'description': 'Get the current stock price of a company', 'parameters': { 'type': 'object', 'properties': { 'company': { 'type': 'string', 'description': 'The name of the company' }, 'required': [ 'company' ] } }, { 'type': 'function', 'function': { 'name': 'get_movie_details', 'description': 'Get details about a movie', 'parameters': { 'type': 'object', 'properties': { 'title': { 'type': 'string', 'description': 'The title of the movie' }, 'required': [ 'title' ] } } } ] </tools> Use the following pydantic model json schema for each tool call you will make: { 'title': 'FunctionCall', 'type': 'object', 'properties': { 'arguments': { 'title': 'Arguments', 'type': 'object', 'name': { 'title': 'Name', 'type': 'string' }, 'required': [ 'arguments', 'name' ] } } } For each function call return a json object with function name and arguments within <tool_call></tool_call> XML tags as follows:\n<tool_call>\n{tool_call}\n</tool_call>",
  "role": "system"
},
{
  "content": "Hi, can you tell me the current stock price of Apple?",
  "role": "human"
},
{
```

HOW AGENTS ARE TRAINED?

```
{
  "content": "You are a function calling AI model. You are provided with function signatures within <tools></tools> XML tags. You may call one or more functions to assist with the user query. Don't make assumptions about what values to plug into functions. Here are the available tools:<tools> [{ 'type': 'function', 'function': { 'name': 'get_stock_price', 'description': 'Get the current stock price of a company', 'parameters': { 'type': 'object', 'properties': { 'company': { 'type': 'string', 'description': 'The name of the company' }, 'required': [ 'company' ] } }, { 'type': 'function', 'function': { 'name': 'get_movie_details', 'description': 'Get details about a movie', 'parameters': { 'type': 'object', 'properties': { 'title': { 'type': 'string', 'description': 'The title of the movie' }, 'required': [ 'title' ] } } } ] </tools> Use the following pydantic model json schema for each tool call you will make: { 'title': 'FunctionCall', 'type': 'object', 'properties': { 'arguments': { 'title': 'Arguments', 'type': 'object', 'name': { 'title': 'Name', 'type': 'string' }, 'required': [ 'arguments', 'name' ] } } } For each function call return a json object with function name and arguments within <tool_call></tool_call> XML tags as follows:\n<tool_call>\n{tool_call}\n</tool_call>",
  "role": "system"
},
{
  "content": "Hi, can you tell me the current stock price of Apple?",
  "role": "human"
},
{
  "content": "<think>Alright, the user asked for the current stock price of Apple. Looking at the tools available, there's a function called get_stock_price which exactly fits this request. I'll need to call this function with the parameter company set to 'Apple' to retrieve the information the user is asking for. This seems like the most straightforward and appropriate action to take.\n</think><tool_call>\n{ 'name': 'get_stock_price', 'arguments': { 'company': 'Apple' } }\n</tool_call>",
  "role": "model"
},
```


HOW AGENTS ARE TRAINED?

```
{
  "content": "You are a function calling AI model. You are provided with function signatures within <tools></tools> XML tags. You may call one or more functions to assist with the user query. Don't make assumptions about what values to plug into functions. Here are the available tools:<tools> [{ 'type': 'function', 'function': { 'name': 'get_stock_price', 'description': 'Get the current stock price of a company', 'parameters': { 'type': 'object', 'properties': { 'company': { 'type': 'string', 'description': 'The name of the company' } }, 'required': [ 'company' ] } } }, { 'type': 'function', 'function': { 'name': 'get_movie_details', 'description': 'Get details about a movie', 'parameters': { 'type': 'object', 'properties': { 'title': { 'type': 'string', 'description': 'The title of the movie' } }, 'required': [ 'title' ] } } } ] </tools> Use the following pydantic model json schema for each tool call you will make: { 'title': 'FunctionCall', 'type': 'object', 'properties': { 'arguments': { 'title': 'Arguments', 'type': 'object', 'name': { 'title': 'Name', 'type': 'string' }, 'required': [ 'arguments', 'name' ] } } } For each function call return a json object with function name and arguments within <tool_call></tool_call> XML tags as follows:\n<tool_call>\n{ 'tool_call': '\n</tool_call>',
  "role": "system"
},
{
  "content": "Hi, can you tell me the current stock price of Apple?",
  "role": "human"
},
{
  "content": "<think>Alright, the user asked for the current stock price of Apple. Looking at the tools available, there's a function called get_stock_price which exactly fits this request. I'll need to call this function with the parameter company set to 'Apple' to retrieve the information the user is asking for. This seems like the most straightforward and appropriate action to take.\n</think><tool_call>\n{ 'name': 'get_stock_price', 'arguments': { 'company': 'Apple' } }\n</tool_call>",
  "role": "model"
},
{
  "content": "<tool_response>\n{ 'stock_price': '$150.75' }\n</tool_response>",
  "role": "tool"
},
}
```


HOW AGENTS ARE TRAINED?

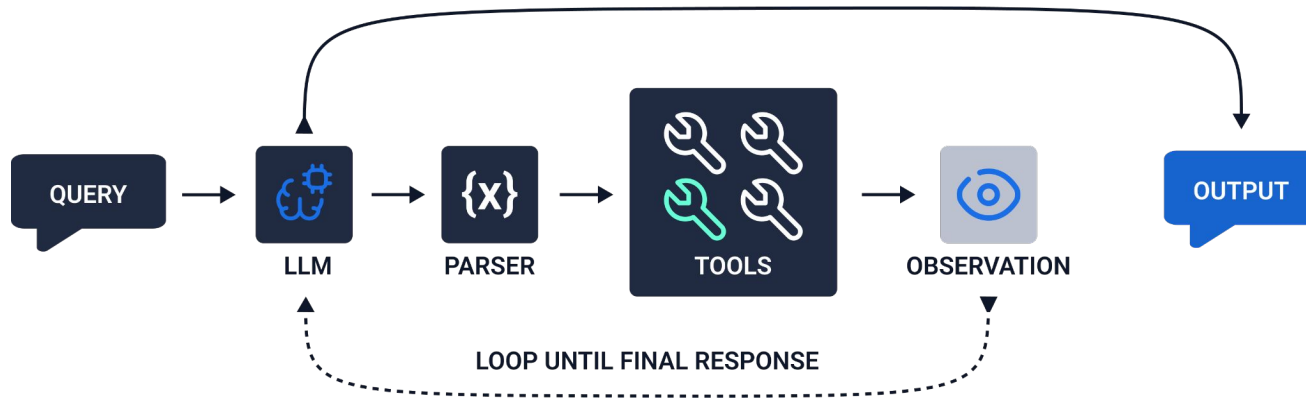
```
{
  "content": "You are a function calling AI model. You are provided with function signatures within <tools></tools> XML tags. You may call one or more functions to assist with the user query. Don't make assumptions about what values to plug into functions. Here are the available tools:<tools> [{ 'type': 'function', 'function': { 'name': 'get_stock_price', 'description': 'Get the current stock price of a company', 'parameters': { 'type': 'object', 'properties': { 'company': { 'type': 'string', 'description': 'The name of the company' } }, 'required': [ 'company' ] } } }, { 'type': 'function', 'function': { 'name': 'get_movie_details', 'description': 'Get details about a movie', 'parameters': { 'type': 'object', 'properties': { 'title': { 'type': 'string', 'description': 'The title of the movie' } }, 'required': [ 'title' ] } } } ] </tools> Use the following pydantic model json schema for each tool call you will make: { 'title': 'FunctionCall', 'type': 'object', 'properties': { 'arguments': { 'title': 'Arguments', 'type': 'object' }, 'name': { 'title': 'Name', 'type': 'string' }, 'required': [ 'arguments', 'name' ] } } For each function call return a json object with function name and arguments within <tool_call></tool_call> XML tags as follows:\n<tool_call>\n{ 'tool_call': }\n</tool_call>",
  "role": "system"
},
{
  "content": "Hi, can you tell me the current stock price of Apple?",
  "role": "human"
},
{
  "content": "<think>Alright, the user asked for the current stock price of Apple. Looking at the tools available, there's a function called get_stock_price which exactly fits this request. I'll need to call this function with the parameter company set to 'Apple' to retrieve the information the user is asking for. This seems like the most straightforward and appropriate action to take.\n</think><tool_call>\n{ 'name': 'get_stock_price', 'arguments': { 'company': 'Apple' } }\n</tool_call>",
  "role": "model"
},
{
  "content": "<tool_response>\n{ 'stock_price': '$150.75' }\n</tool_response>",
  "role": "tool"
},
{
  "content": "The current stock price of Apple is $150.75.",
  "role": "model"
},
}
```

HOW AGENTS ARE TRAINED?

```
{
  "content": "You are a function calling AI model. You are provided with function signatures within <tools></tools> XML tags. You may call one or more functions to assist with the user query. Don't make assumptions about what values to plug into functions. Here are the available tools:<tools> [{ 'type': 'function', 'function': { 'name': 'get_stock_price', 'description': 'Get the current stock price of a company', 'parameters': { 'type': 'object', 'properties': { 'company': { 'type': 'string', 'description': 'The name of the company' } }, 'required': [ 'company' ] } }, { 'type': 'function', 'function': { 'name': 'get_movie_details', 'description': 'Get details about a movie', 'parameters': { 'type': 'object', 'properties': { 'title': { 'type': 'string', 'description': 'The title of the movie' } }, 'required': [ 'title' ] } } } ]</tools> Use the following pydantic model json schema for each tool call you will make: { 'title': 'FunctionCall', 'type': 'object', 'properties': { 'arguments': { 'title': 'Arguments', 'type': 'object', 'name': { 'title': 'Name', 'type': 'string' }, 'required': [ 'arguments', 'name' ] } } }</tools> For each function call return a json object with function name and arguments within <tool_call></tool_call> XML tags as follows:\n<tool_call>\n{ 'tool_call': }\n</tool_call>",
  "role": "system"
},
{
  "content": "Hi, can you tell me the current stock price of Apple?",
  "role": "human"
},
{
  "content": "<think>Alright, the user asked for the current stock price of Apple. Looking at the tools available, there's a function called get_stock_price which exactly fits this request. I'll need to call this function with the parameter company set to 'Apple' to retrieve the information the user is asking for. This seems like the most straightforward and appropriate action to take.\n</think><tool_call>\n{ 'name': 'get_stock_price', 'arguments': { 'company': 'Apple' } }\n</tool_call>",
  "role": "model"
},
{
  "content": "<tool_response>\n{ 'stock_price': '$150.75' }\n</tool_response>",
  "role": "tool"
},
{
  "content": "The current stock price of Apple is $150.75.",
  "role": "model"
},
}
```

3570 rows

HOW IT WORKS?



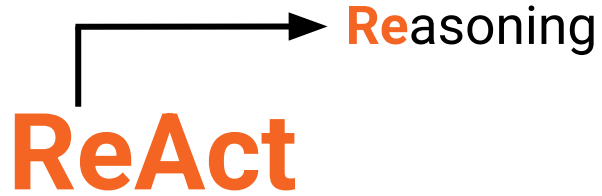
INTRODUCTION

REACT APPROACH

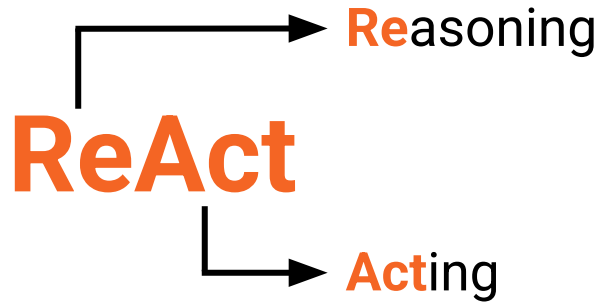
ReAct

INTRODUCTION

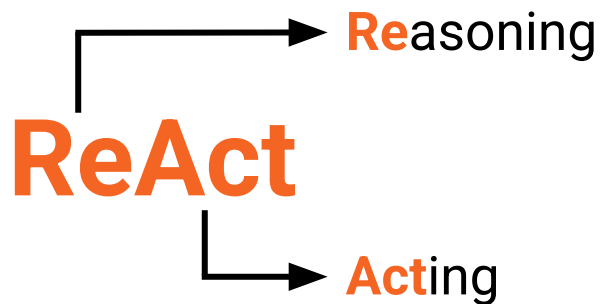
REACT APPROACH



REACT APPROACH



REACT APPROACH



It's a prompting technique that encourages the model to think **step-by-step** and **interleave** actions between reasoning steps.

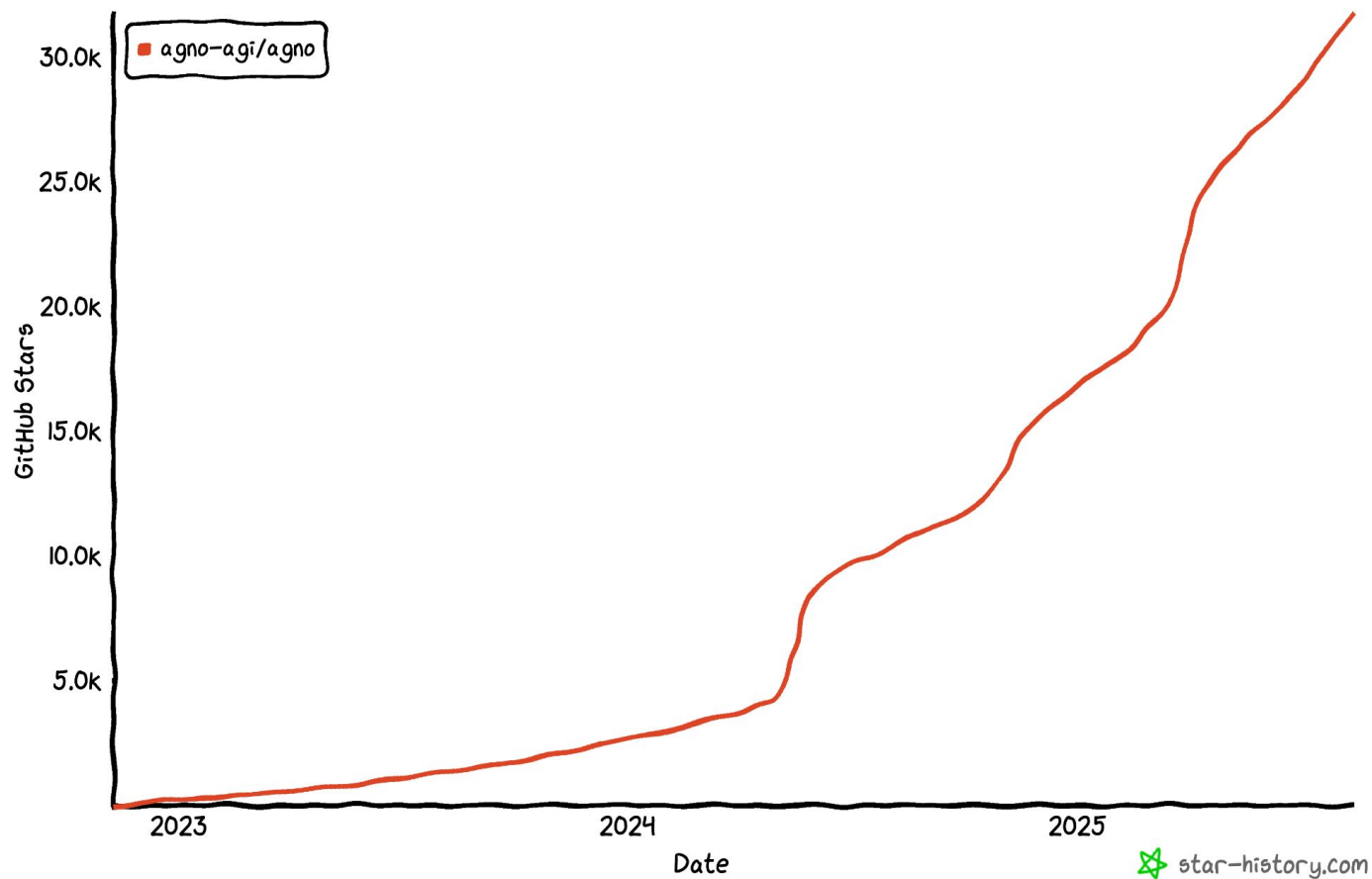


Hugging Face

HANDS-ON

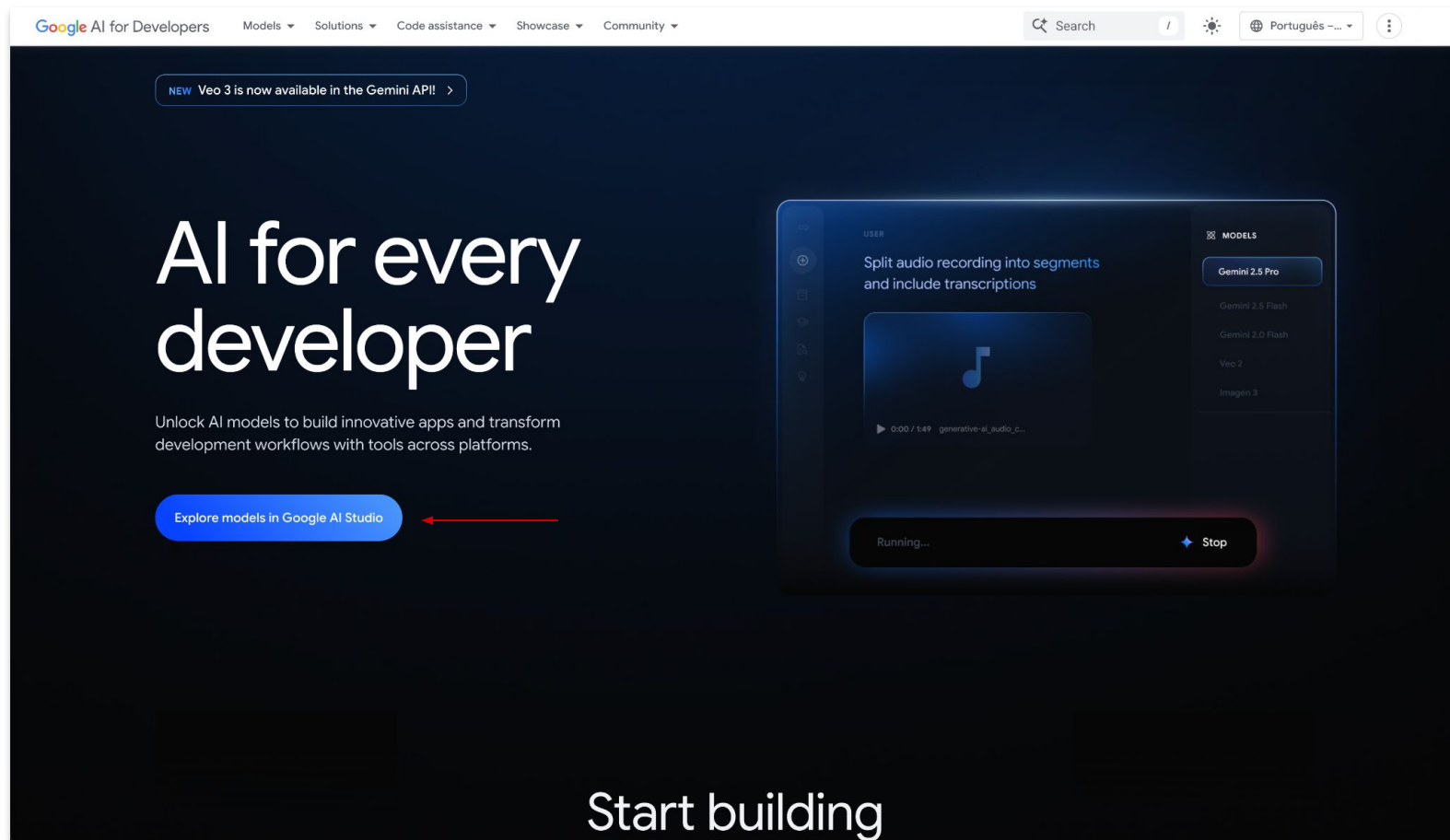
#1

Star History



The Gemini logo features the word "Gemini" in a sans-serif font. The "G" is blue, while the "emini" part has a blue-to-purple gradient. Above the dot of the "i" is a purple four-pointed star. The final dot of the "i" is red.

Gemini



Google AI for Developers

Models Solutions Code assistance Showcase Community

Search / Portuguese

NEW Veo 3 is now available in the Gemini API! >

AI for every developer

Unlock AI models to build innovative apps and transform development workflows with tools across platforms.

Explore models in Google AI Studio

USER

Split audio recording into segments and include transcriptions

0:00 / 1:49 generative-ai_audio_c...

MODELS

- Gemini 2.5 Pro
- Gemini 2.5 Flash
- Gemini 2.0 Flash
- Veo 2
- Imagen 3

Running... Stop

Start building

Google AI Studio

Chat

Stream

Generate media

Build

History

Enable saving

Chat prompt

Google AI Studio

Generate Python code for a simple calculator app →

Run

What's new



URL context tool
Fetch information from web links



Native speech generation
Generate high quality text to speech with Gemini



Live audio-to-audio dialog
Try Gemini's natural, real-time dialog with audio and video inputs



Native image generation
Interleaved text-and-image generation with Gemini 2.0 Flash

Google AI models may make mistakes, so double-check outputs.

Get API key

Studio Dashboard Documentation

Run settings

Gemini 2.5 Pro

Token count 0 / 1,048,576

Temperature 1

Media resolution Default

Thinking

Thinking mode

Set thinking budget

Tools

Structured output

Code execution

Function calling

Grounding with Google Search

URL context

Advanced settings

Google AI Studio

API keys

Usage & Billing

Changelog

Get API key

Studio

Dashboard

Documentation

Chaves de API

Testar rapidamente a API Gemini

[Guia de início rápido da API](#)

```
<> Code
curl "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent" \
-H 'Content-Type: application/json' \
-H 'X-goog-api-key: GEMINI_API_KEY' \
-X POST \
-d '{
  "contents": [
    {
      "parts": [
        {
          "text": "Explain how AI works in a few words"
        }
      ]
    }
  ]
}'
```

Suas chaves de API estão listadas abaixo. Também é possível visualizar e gerenciar seu projeto e as chaves de API no Google Cloud.

Look up API Key for project

Número do projeto	Nome do projeto	Chave de API	Criação	Plano
Crie uma chave de API para ver seus projetos				

Use as chaves de API com segurança. Não as compartilhe nem as incorpore em códigos públicos. O uso da API Gemini em um projeto com faturamento ativado está sujeito aos [preços do modelo de pagamento por uso](#).

Google AI Studio

Get API keys

Usage & Billing

Changelog

Get API key

Studio

Dashboard

Documentation

Chaves de API

+ Criar chave de API

Testar rapidamente a API Gemini

[Guia de início rápido da API](#)

<> Code

```
curl "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent" \
-H "Content-Type: application/json" \
-H "X-goog-api-key: GEMINI_API_KEY" \
-X POST \
-d '{
  "contents": [
    {
      "parts": [
        {
          "text": "Explain how AI works"
        }
      ]
    }
  ]
}'
```

Criar chave de API

Selecione um dos seus projetos atuais do Google Cloud

Search Google Cloud projects

⇒ Criar uma chave de API em um projeto atual

Suas chaves de API estão listadas abaixo. Também é possível visualizar e gerenciar seu projeto e as chaves de API no Google Cloud.

Look up API Key for project

Nome do projeto	Nome do projeto	Chave de API	Criação	Plano
Crie uma chave de API para ver seus projetos				

View status

Use as chaves de API com segurança. Não as compartilhe nem as incorpore em códigos públicos. O uso da API Gemini em um projeto com faturamento ativado está sujeito aos [preços do modelo de pagamento por uso](#).

Google AI Studio

Get API keys

Usage & Billing

Changelog

Get API key

Studio

Dashboard

Documentation

Chaves de API

Testar rapidamente a API Gemini

[Guia de início rápido da API](#)

<> Code

```
curl "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent" \
-H "Content-Type: application/json" \
-H "X-goog-api-key: GEMINI_API_KEY" \
-X POST \
-d '{
  "contents": [
    {
      "parts": [
        {
          "text": "Explain how AI works"
        }
      ]
    }
  ]
}'
```

Criar chave de API

Selecione um dos seus projetos atuais do Google Cloud

Search Google Cloud projects

 Gemini API gen-lang-client-0581720549

Suas chaves de API estão listadas abaixo. Também é possível visualizar e gerenciar seu projeto e as chaves de API no Google Cloud.

Look up API Key for project

Número do projeto	Nome do projeto	Chave de API	Criação	Plano
Crie uma chave de API para ver seus projetos				

Use as chaves de API com segurança. Não as compartilhe nem as incorpore em códigos públicos. O uso da API Gemini em um projeto com faturamento ativado está sujeito aos [preços do modelo de pagamento por uso](#).

View status

Google AI Studio

Get API keys

Usage & Billing

Changelog

Get API key

Studio

Dashboard

Documentation

Chaves de API

Testar rapidamente a API Gemini

[Guia de início rápido da API](#)

<> Code

```
curl "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent" \
-H "Content-Type: application/json" \
-H "X-goog-api-key: GEMINI_API_KEY" \
-X POST \
-d '{
  "contents": [
    {
      "parts": [
        {
          "text": "Explain how AI works"
        }
      ]
    }
  ]
}'
```

Criar chave de API

Selecione um dos seus projetos atuais do Google Cloud

Search Google Cloud projects

Criar uma chave de API em um projeto atual

Suas chaves de API estão listadas abaixo. Também é possível visualizar e gerenciar seu projeto e as chaves de API no Google Cloud.

Look up API Key for project

Número do projeto	Nome do projeto	Chave de API	Criação	Plano
Crie uma chave de API para ver seus projetos				

[View status](#)

Use as chaves de API com segurança. Não as compartilhe nem as incorpore em códigos públicos. O uso da API Gemini em um projeto com faturamento ativado está sujeito aos [preços do modelo de pagamento por uso](#).

Google AI Studio

Get API keys

Usage & Billing

Changelog

Get API key

Studio

Dashboard

Documentation

Chaves de API

+ Criar chave de API

Testar rapidamente a API Gemini

[Guia de início rápido da API](#)

<> Code

```
curl "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash/generateContent" \
  -H "Content-Type: application/json" \
  -H "X-goog-api-key: GEMINI_API_KEY" \
  -X POST \
  -d '{
    "contents": [
      {
        "parts": [
          {
            "text": "E"
          }
        ]
      }
    ]
  }'
```

Chave de API gerada

Use suas chaves de API com segurança. Não compartilhe nem as incorpore em códigos que o público possa ver.

AlzaSyAMaONxG9FojqVGfNj9X3T9_pv-vzErmTE

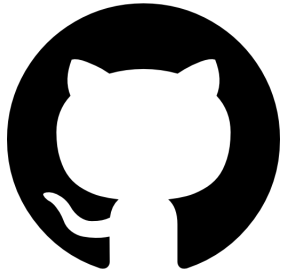
Copiar

Suas chaves de API estão listadas abaixo. Também é possível visualizar e gerenciar seu projeto e as chaves de API no Google Cloud.

Look up API Key for project

Número do projeto	Nome do projeto	Chave de API	Criação	Plano
...1456	Gemini API	...rmTE	25 de ago. de 2025	Configurar o faturamento Ver dados de uso

Use as chaves de API com segurança. Não as compartilhe nem as incorpore em códigos públicos. O uso da API Gemini em um projeto com faturamento ativado está sujeito aos [preços do modelo de pagamento por uso](#).



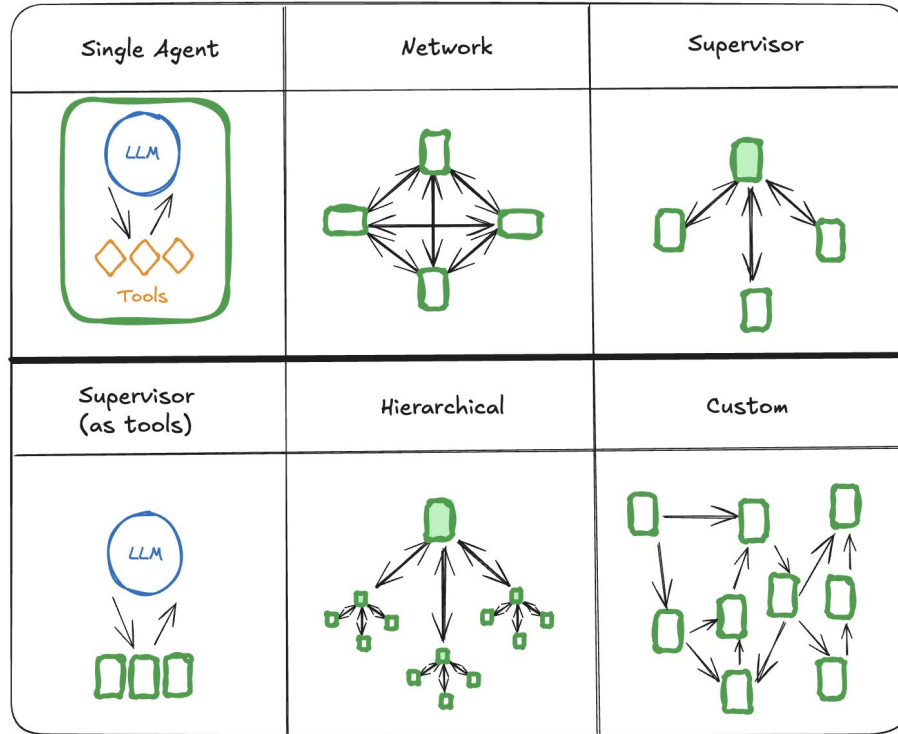
https://github.com/conect2ai/2025-UNIBS-Courses/course_agents/code/

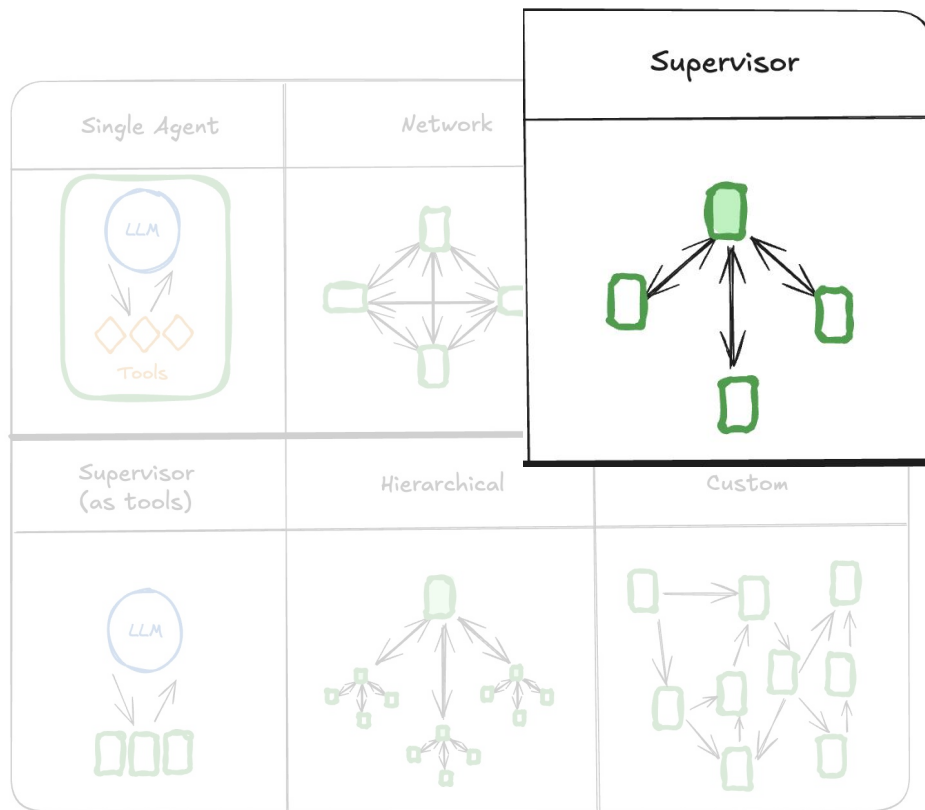
01_HandsOn.ipynb

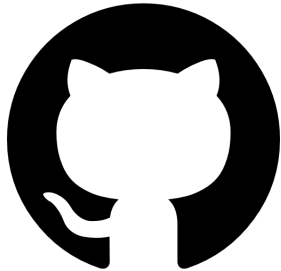


HANDS-ON

#2





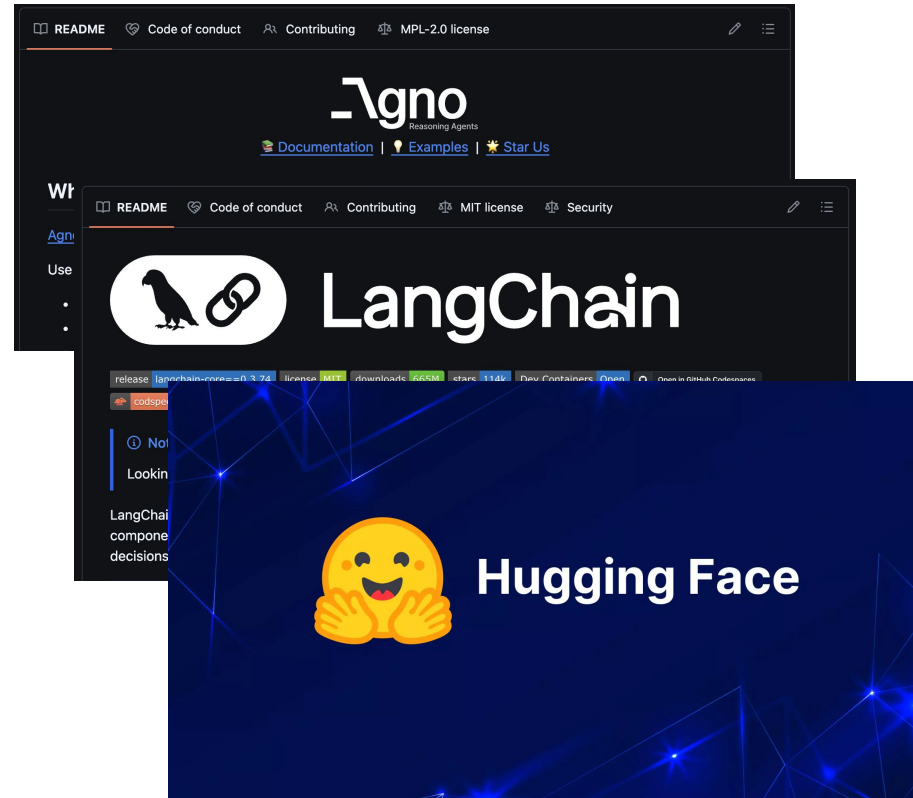
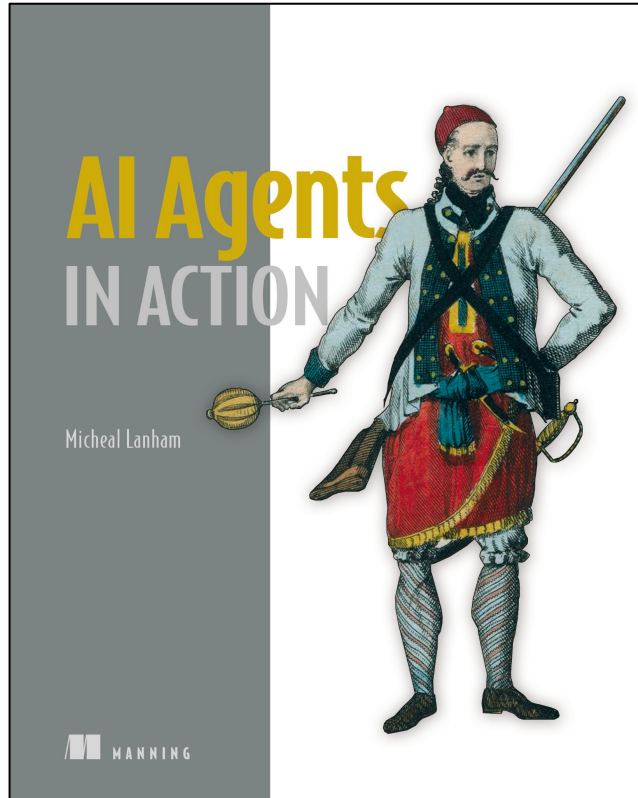


https://github.com/conect2ai/2025-UNIBS-Courses/course_agents/code/

02_HandsOn.ipynb

CONCLUSION

REFERENCES



THANK YOU

matheus.diniz.122@ufrn.edu.br